

ОБОБЩИТЕЛНА ЗАДАЧА за Пикасо

„Learn the rules like a pro, so you can break them like an artist.“ – Pablo Picasso

Пабло Пикасо е най-значимият художник на XX век, създател на кубизма. Той е най-скъпият и краденият художник. Когато вижда ужасите от войната, рисува най-известния си шедьовър "Герника" за бомбардировките над мирното испанско градче. Картината е изцяло черно-бяла, без други цветове и се счита за най-трогателната и силна антивоенна картина в историята. За нея, художникът казва "Изкуството не е направено, за да декорира стените на апартаментите. То е мощно оръжие за защита срещу враговете".



Негови известни цитати:

- * "Научи правилата като професионалист, за да ги разбиваш като артист."
 - * "Всяко дете е творец. Проблемът е как да се запази такъв, когато порасне."
 - * "Изкуството е лъжа, която ни помага да осъзнаем истината."
 - * "Лошите художници копират. Добрите крадат."
 - * "Смисълът на живота е да намериш своята дарба. Значението на живота е да я предадеш."
- " Има художници, които превръщат слънцето в жълто петно, но има други, които с помощта на тяхното изкуство и на тяхната интелигентност превръщат жълто петно в слънце."

- * Всичко, което човек може да си представи, е истинско."
- * "Светът няма смисъл, защо тогава аз да рисувам картини, които имат?"
- * "Няма абстрактно изкуство. Винаги започваш с нещо. След това може да премахнеш всички следи от реалността."
- * "Отне ми четири години да се науча да рисувам като Рафаел, но цял живот, за да рисувам като дете."
- * "Когато бях малък, майка ми казваше:
Ако станеш войник, ще стигнеш до генерал.
Ако станеш свещеник, ще бъдеш папа.
Аз исках да бъда художник и станах Пикасо. "
- * "Винаги правя това, което не мога, за да мога да се науча как да го правя."
- * "Ако нарисувам препускащ кон, ти може да не видиш коня, но ще усетиш препускането."
- * "Изкуството отмива от душата прахта от ежедневието."
- * "Да имитираш другите е неизбежно. Позорно е, когато започнеш да имитираш себе си."
- * "Бог е художник. Той е създал жирафа, слона и котката."
- "Компютрите са безполезни. Те могат да ти дадат само отговори."
- * "Не се страхувай от перфектността - никога няма да я стигнеш."

Нека изготвим демонстрационна програма на тема за Пикасо с включване на максимално много от изучените функционалности:

```

Microsoft Visual Studio Debug Console

ПАБЛО ПИКАСО

=== Всички картини в галерията ===
Общ брой: 6

=== НАЙ-СКЪПА КАРТИНА ===
"Герника", цена: 200 000 000,00 лв.

=== КАРТИНИ ПО АЗБУЧЕН РЕД ===
■ Герника
■ Животът
■ Младият акробат
■ Плачещата жена
■ Старият китарист
■ Тримата музиканти

=== КРАТКИ ОПИСАНИЯ ===
▶ "Старият китарист", (1903) / Ранен период (1900-1909). Меланхолични тонове, социална тематика, емоционална дълбочина.
▶ "Животът", (1903) / Ранен период (1900-1909). Меланхолични тонове, социална тематика, емоционална дълбочина.
▶ "Младият акробат", (1905) / Ранен период (1900-1909). Меланхолични тонове, социална тематика, емоционална дълбочина.
▶ "Герника", (1937) / Аналитичен кубизъм. Иновативен подход, революция в изкуството, геометрични форми.
▶ "Плачещата жена", (1937) / Синтетичен кубизъм. Иновативен подход, революция в изкуството, геометрични форми.
▶ "Тримата музиканти", (1921) / Синтетичен кубизъм. Иновативен подход, революция в изкуството, геометрични форми.

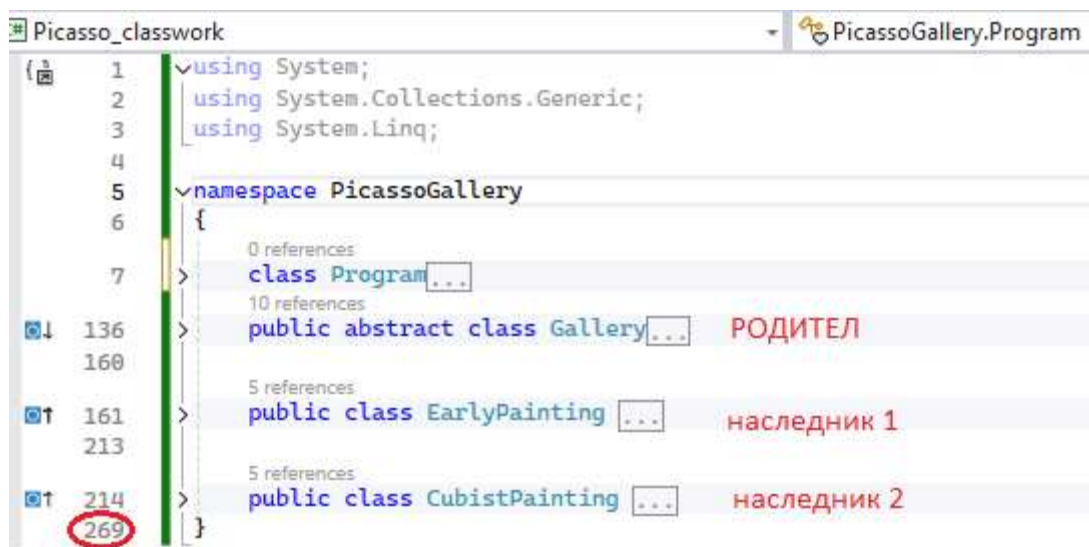
Потърсете в архива: Герника
=== СПИСКЪК СЪС СЪОТВЕТСТВАЩИ КАРТИНИ ===
=== Подробни характеристики (Кубистичен период) ===
Заглавие: "Герника"
Художник: Пабло Пикасо
Период: Кубистичен и по-късен период (1910-1973)
Фаза на кубизма: Аналитичен
Година на създаване: 1937
Размери: 349 x 776 cm
Местоположение: Музей София, Мадрид
Оценявана стойност: 200 000 000,00 лв.
Стил: Кубизъм
Характеристики на периода: Деформация на форми, множествена перспектива, геометрични форми, колаж

<<<<<< ARS LONGA, VITA BREVIS >>>>>>

D:\Exercises\11 class\Inheritance\Picasso_classwork\bin\Debug\net8.0\Picasso_classwork.exe (process 9616) exited with
de 0.

```

Основните класове, които ни трябва:



Започваме от абстрактния родител:

```
public abstract class Gallery
{
    10 references                                     родител + наследници
    public string Title { get; protected set; } // Всички четат, само protected пишат
    2 references
    public string Artist { get; } = "Пабло Пикасо"; // read-only + начална стойност
    4 references
    protected internal string Style { get; set; }
    защитен + достъпен в Main()
    protected internal static int counter = 0; static - принадлежи на класа

    2 references
    protected Gallery(string title, string style) ctor
    {
        counter++; +1 при всяко създаване на нов обект
        Title = string.IsNullOrEmpty(title) ? "Неизвестна картина" : title;
        Style = style;
    }

    3 references
    public virtual void Description() незадължителен, има тяло
    {
        Console.WriteLine($"{Title} в стил {Style}");
    }

    3 references
    public abstract void Details(); задължителен

    4 references                                     ТЕКСТ
    public abstract string PeriodCharacteristics(); форматиране / подреждане
}
```

Слагаме дори валидация в конструктора.

Създаваме 2 наследника според 2 големи периода на неговото изкуство: ранен и кубизъм.

С **prop + Tab x 2** създаваме auto property. Дописваме с private за по-голяма защита, което е широко разпространен синтаксис (всеки може да прочете, но само класа да запише).

```
public class EarlyPainting : Gallery
{
    3 references                                     read-only + write only
    public int YearCreated { get; private set; }
    2 references
    public string Dimensions { get; private set; }
    2 references
    public string Location { get; private set; }
    3 references
    public decimal EstimatedValue { get; private set; }
    3 references
    public string Period { get; private set; }
}
```


Конструкторът има няколко интересни момента: проверка за грешен период, който ще хвърли грешка, валидация, инициализация:

```
public EarlyPainting(string title, int year, string dimensions,
                    string location, decimal value, string style)
    : base(title, style) наследяване
{
    if (year < 1900 || year > 1909)
        throw new ArgumentException("Ранният период е 1900-1909 г.");

    YearCreated = YearCreated < 0 ? 1900 : year; валидация
    Dimensions = dimensions;
    Location = location;
    EstimatedValue = value;
    Period = "Ранен период (1900-1909)"; инициализиране
}
```

Редактираме и няколко метода:

```
public override void Description()
{
    Console.WriteLine($"{Title}\n", ({YearCreated}) / {Period}. {GetShortDescription()}");
}

2 references
public override void Details()
{
    Console.WriteLine("=== Подробни характеристики ===");
    Console.WriteLine($"Заглавие: {Title}");
    Console.WriteLine($"Художник: {Artist}");
    Console.WriteLine($"Период: {Period}");
    Console.WriteLine($"Година на създаване: {YearCreated}");
    Console.WriteLine($"Размери: {Dimensions}");
    Console.WriteLine($"Местоположение: {Location}"); Currency - точки
    Console.WriteLine($"Оценявана стойност: {EstimatedValue:C}");
    Console.WriteLine($"Стил: {Style}");
    Console.WriteLine($"Характеристики на периода: {PeriodCharacteristics()}");
    Console.WriteLine();
}

2 references
public override string PeriodCharacteristics()
{
    return "Синя и Розова епоха, меланхолия, социални теми, цигани, артисти";
}

1 reference
private string GetShortDescription() собствен метод
{
    return "Меланхолични тонове, социална тематика, емоционална дълбочина.";
}
```

Другият наследник е много сходен:

```
public class CubistPainting : Gallery
{
    3 references
    public int YearCreated { get; private set; }
    2 references
    public string Dimensions { get; private set; }
    2 references
    public string Location { get; private set; }
    3 references
    public decimal EstimatedValue { get; private set; }
    2 references
    public string Period { get; private set; }
    3 references
    public string CubistPhase { get; private set; } // Аналитичен/Синтетичен кубизъм

    3 references
    public CubistPainting(string title, int year, string dimensions,
                           string location, decimal value, string style, string cubistPhase)
        : base(title, style)
    {
        if (year < 1910 || year > 1973)
            throw new ArgumentException("Кубистичният период е 1910-1973 г.");

        YearCreated = year;
        Dimensions = dimensions;
        Location = location;
        EstimatedValue = value;
        Period = "Кубистичен и по-късен период (1910-1973)";
        CubistPhase = cubistPhase;
    }
}
```

Методите също, с незначителни разлики:

```
public override void Description()
{
    Console.WriteLine($">\ \"{Title}\", ({YearCreated}) / {CubistPhase} кубизъм. {GetShortDescription()}");
}

2 references
public override void Details()
{
    Console.WriteLine("=== Подробни характеристики (Кубистичен период) ===");
    Console.WriteLine($"Заглавие: \"{Title}\"");
    Console.WriteLine($"Художник: {Artist}");
    Console.WriteLine($"Период: {Period}");
    Console.WriteLine($"Фаза на кубизма: {CubistPhase}");
    Console.WriteLine($"Година на създаване: {YearCreated}");
    Console.WriteLine($"Размери: {Dimensions}");
    Console.WriteLine($"Местоположение: {Location}");
    Console.WriteLine($"Оценявана стойност: {EstimatedValue:C}");
    Console.WriteLine($"Стил: {Style}");
    Console.WriteLine($"Характеристики на периода: {PeriodCharacteristics()}");
    Console.WriteLine();
}

2 references
public override string PeriodCharacteristics()
{
    return "Деформация на форми, множествена перспектива, геометрични форми, колаж";
}

1 reference
private string GetShortDescription()
{
    return "Иновативен подход, революция в изкуството, геометрични форми.";
}
```

Интересно е как да си подредим Main() – т.к. искаме да спазим добрите практики и да отделим използваният код извън Main(), а в него да останат само имената на методите, списъкът трябва да бъде достъпен за тях, т.е. извън Main(), но разбира се, в рамките на класа.

```

0 references
class Program static List над Main(), за да бъде достъпен от методите
{
    static List<Gallery> allPaintings = new List<Gallery>();

    0 references
    static void Main()...

    1 reference
    static void AddPaintings()... добавя картини

    1 reference
    static void DisplayHeading()... заглавие

    1 reference
    static void MostExpensive()... най-скъпа картина

    1 reference
    private static void AlphabeticalOrder(List<Gallery> allPaintings)...

    1 reference
    static void ShowDescriptions()...

    1 reference
    static void ShowDetails(string searchTitle)...
```

Main() прихваща грешки с try-catch, освен това, понеже списъкът е достъпен, ние имаме възможност да го променяме чрез .Add(), .Insert(), .Remove() – списъците са гъвкави, а конструкторът предоставя удобство при въвеждането на стойностите.


```

static void Main()
{
    try Обичайният Main()-код стои в частта try
    {
        AddPaintings(); .Add() към списъка, или .Insert(), или .Remove()
        allPaintings.Add(new CubistPainting(
            "Тримата музиканти", 1921, "203.2 x 187.9 cm",
            "Музей за модерно изкуство, Ню Йорк", 120000000m, "Кубизъм", "Синтетичен"));

        DisplayHeading();

        MostExpensive();

        AlphabeticalOrder(allPaintings);
        ShowDescriptions();

        Console.Write("Потърсете в архива: ");
        string searchTitle = Console.ReadLine();
        ShowDetails(searchTitle);
    }
    catch (Exception ex)
    {
        Console.WriteLine("Грешка: " + ex.Message);
    }
    finally
    {
        "Животът е кратък, изкуството вечно"
        Console.WriteLine("<<<<<< ARS LONGA, VITA BREVIS >>>>>>");
        (латински цитат само с главни букви)
    }
    Console.ReadKey();
}

```

Нека разгледаме кодовете на отделните методи:

```
static void AddPaintings() // Не връща списък, само добавя елементи
{
    allPaintings.Add(new EarlyPainting(
        "Старият китарист", 1903, "122.9 x 82.6 cm", decimal
        "Чикагски институт по изкуствата", 600000000m, M "Синя епоха"
    ));

    allPaintings.Add(new EarlyPainting(
        "Животът", 1903, "196.5 x 129.2 cm",
        "Музей на изкуството, Кливланд", 500000000m, "Синя епоха"
    ));

    allPaintings.Add(new EarlyPainting(
        "Младият акробат", 1905, "93.3 x 80.6 cm",
        "Музей за модерно изкуство, Ню Йорк", 450000000m, "Розова епоха"
    ));

    allPaintings.Add(new CubistPainting(
        "Герника", 1937, "349 x 776 cm",
        "Музей София, Магрик", 2000000000m, "Кубизъм", "Аналитичен"
    ));
    // Демонстрираме грешка в съответствие година - период
    /*
    allPaintings.Add(new CubistPainting(
        "Авиньонските момичета", 1907, "243.9 x 233.7 cm",
        "Музей за модерно изкуство, Ню Йорк", 1500000000m, "Прото-кубизъм", "Аналитичен"
    ));*/

    allPaintings.Add(new CubistPainting(
        "Плачещата жена", 1937, "60 x 49 cm",
        "Тейт Модърн, Лондон", 800000000m, "Кубизъм", "Синтетичен"
    ));
}
```

Заглавна лента:

```
static void DisplayHeading()
{
    Console.WriteLine("
    Console.WriteLine("
    Console.WriteLine("
    Console.WriteLine("=== Всички картини в галерията ===");
    Console.WriteLine($"Общ брой: {class static Gallery.counter}\n");
}
```

Максимална стойност .Max, но максимална от група .MaxBy. Аналогични са .Average(), .Min...

```

static void MostExpensive()
{
    Console.WriteLine("=== НАЙ-СКЪПА КАРТИНА ===");
    var mostExpensive = allPaintings.MaxBy(p => GetPaintingValue(p));
    Console.WriteLine($"{mostExpensive.Title}\n", цена: {GetPaintingValue(mostExpensive):C});

    static decimal GetPaintingValue(Gallery painting)
    {
        if (painting is EarlyPainting early) наследник 1
            return early.EstimatedValue;
        else if (painting is CubistPainting cubist) наследник 2
            return cubist.EstimatedValue;
        return 0;
    }
    Console.WriteLine();
}

```

Подредба -> а в намаляващ ред е Descending.

```

private static void AlphabeticalOrder(List<Gallery> allPaintings)
{
    Console.WriteLine("=== КАРТИНИ ПО АЗБУЧЕН РЕД ===");
    var orderedPaintings = allPaintings.OrderBy(p => p.Title).ToList();
    foreach (var painting in orderedPaintings)
    {
        Console.WriteLine($" ■ {painting.Title}");
    }
    Console.WriteLine();
}

```

Обхождане с извикване на метод от наследника:

```

static void ShowDescriptions()
{
    Console.WriteLine("=== КРАТКИ ОПИСАНИЯ ===");
    foreach (var painting in allPaintings)
    {
        painting.Description();
    }
    Console.WriteLine();
}

```

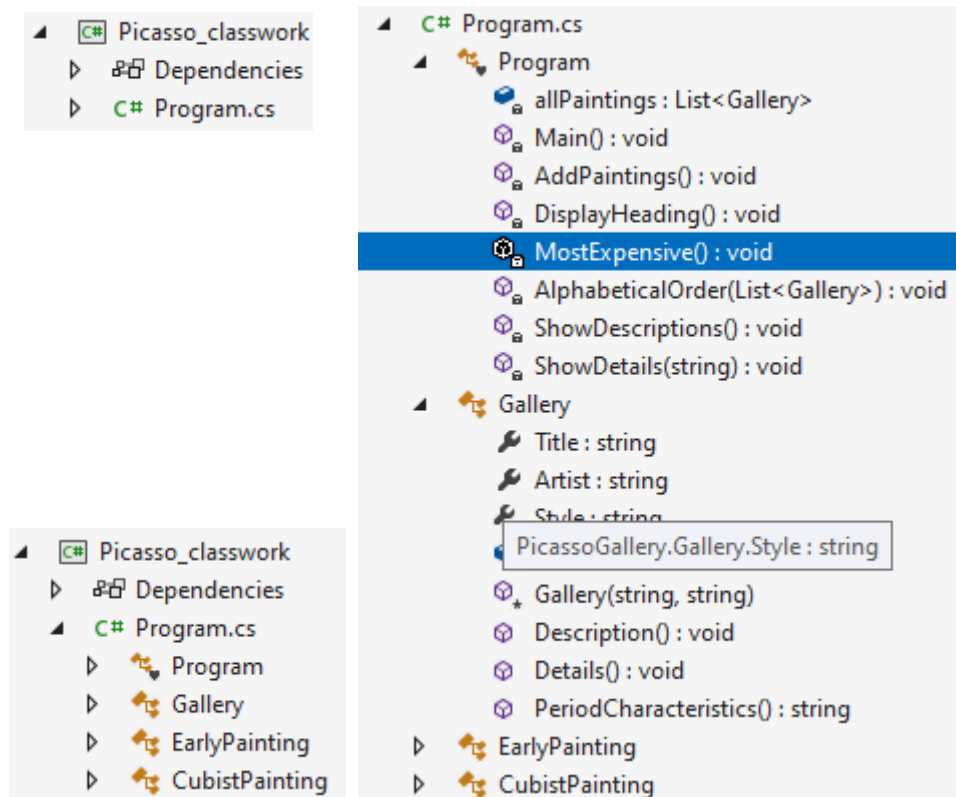
Една от многото търсачки – може да използваме ==, StartsWith, EndsWith и други.

```

static void ShowDetails(string searchTitle)
{
    Console.WriteLine("=== СПИСКЪК СЪС СЪОТВЕТСТВАЩИ КАРТИНИ ===");
    var searchResults = allPaintings.Where(p => p.Title.Contains(searchTitle));
    foreach (var painting in searchResults)
    {
        painting.Details();
    }
}

```

Последно, нека разгледаме и дали VS ще изобрази по някакъв йерархичен начин класовете и методите, които сме използвали. Разбира се, още по-добрият вариант е класовете да бъдат в отделни файлове. От изгледа, с който сме свикнали, нека разлистим Program.cs:



Може да прочетем какъв тип връща метода, какви са свойствата и при клик върху елемент, ни отвежда директно в онази част от кода, в която се използва.

```
static void MostExpensive()
{
    Console.WriteLine("=== НАЙ-СКЪПА КАРТИНА ===");

    var mostExpensive = allPaintings.MaxBy(p => GetPaintingValue(p));

    Console.WriteLine($"{mostExpensive.Title}\", цена: {GetPaintingValue(mostExpensive):C}");
}
```

Това ни спестява хаотично скрулиране и търсене при дълги програми.

Целият код:

```
using System;
using System.Collections.Generic;
using System.Linq;

namespace PicassoGallery
{
    class Program
    {
        static List<Gallery> allPaintings = new List<Gallery>();
    }
}
```



```

static void Main()
{
    try
    {
        AddPaintings();
        allPaintings.Add(new CubistPainting(
            "Тримата музиканти", 1921, "203.2 x 187.9 cm",
            "Музей за модерно изкуство, Ню Йорк", 1200000000m, "Кубизъм",
            "Синтетичен"));

        DisplayHeading();

        MostExpensive();

        AlphabeticalOrder(allPaintings);
        ShowDescriptions();

        Console.Write("Потърсете в архива: ");
        string searchTitle = Console.ReadLine();
        ShowDetails(searchTitle);
    }
    catch (Exception ex)
    {
        Console.WriteLine("Грешка: " + ex.Message);
    }
    finally
    {
        Console.WriteLine("<<<<<< ARS LONGA, VITA BREVIS >>>>>>");
    }
    Console.ReadKey();
}

static void AddPaintings()
{
    allPaintings.Add(new EarlyPainting(
        "Старият китарист", 1903, "122.9 x 82.6 cm",
        "Чикагски институт по изкуствата", 600000000m, "Синя епоха"
    ));

    allPaintings.Add(new EarlyPainting(
        "Животът", 1903, "196.5 x 129.2 cm",
        "Музей на изкуството, Кливланд", 500000000m, "Синя епоха"
    ));

    allPaintings.Add(new EarlyPainting(
        "Младият акробат", 1905, "93.3 x 80.6 cm",
        "Музей за модерно изкуство, Ню Йорк", 450000000m, "Розова епоха"
    ));

    allPaintings.Add(new CubistPainting(
        "Герника", 1937, "349 x 776 cm",
        "Музей София, Мадрид", 2000000000m, "Кубизъм", "Аналитичен"
    ));
    // Демонстрираме грешка в съответствие година – период
    /*
    allPaintings.Add(new CubistPainting(
        "Авиньонските момичета", 1907, "243.9 x 233.7 cm",

```

```

        "Музей за модерно изкуство, Ню Йорк", 1500000000m, "Прото-кубизъм",
        "Аналитичен"
    ));*/

    allPaintings.Add(new CubistPainting(
        "Плачещата жена", 1937, "60 x 49 см",
        "Тейт Модърн, Лондон", 800000000m, "Кубизъм", "Синтетичен"
    ));
}

static void DisplayHeading()
{
    Console.WriteLine(" ");
    Console.WriteLine(" ПАБЛО ПИКАСО ");
    Console.WriteLine(" ");
    Console.WriteLine("=== Всички картини в галерията ===");

    Console.WriteLine($"Общ брой: {Gallery.counter}\n");
}

static void MostExpensive()
{
    Console.WriteLine("=== НАЙ-СКЪПА КАРТИНА ===");

    var mostExpensive = allPaintings.MaxBy(p => GetPaintingValue(p));

    Console.WriteLine($"\"{mostExpensive.Title}\", цена:
{GetPaintingValue(mostExpensive):C}");

    static decimal GetPaintingValue(Gallery painting)
    {
        if (painting is EarlyPainting early)
            return early.EstimatedValue;
        else if (painting is CubistPainting cubist)
            return cubist.EstimatedValue;
        return 0;
    }
    Console.WriteLine();
}

private static void AlphabeticalOrder(List<Gallery> allPaintings)
{
    Console.WriteLine("=== КАРТИНИ ПО АЗБУЧЕН РЕД ===");
    var orderedPaintings = allPaintings.OrderBy(p => p.Title).ToList();
    foreach (var painting in orderedPaintings)
    {
        Console.WriteLine($" ■ {painting.Title}");
    }
    Console.WriteLine();
}

static void ShowDescriptions()
{
    Console.WriteLine("=== КРАТКИ ОПИСАНИЯ ===");
    foreach (var painting in allPaintings)
    {
        painting.Description();
    }
}

```

```

        Console.WriteLine();
    }

    static void ShowDetails(string searchTitle)
    {
        Console.WriteLine("=== СПИСЪК СЪС СЪОТВЕТСТВАЩИ КАРТИНИ ===");
        var searchResults = allPaintings.Where(p =>
п.Title.Contains(searchTitle));
        foreach (var painting in searchResults)
        {
            painting.Details();
        }
    }
}

public abstract class Gallery
{
    public string Title { get; protected set; } // всички четат, само protected
    public string Artist { get; } = "Пабло Пикасо"; // read-only
    protected internal string Style { get; set; }

    protected internal static int counter = 0;

    protected Gallery(string title, string style)
    {
        counter++;
        Title = string.IsNullOrEmpty(title) ? "Неизвестна картина" : title;
        Style = style;
    }

    public virtual void Description()
    {
        Console.WriteLine($"{Title} в стил {Style}");
    }

    public abstract void Details();

    public abstract string PeriodCharacteristics();
}

public class EarlyPainting : Gallery
{
    public int YearCreated { get; private set; }
    public string Dimensions { get; private set; }
    public string Location { get; private set; }
    public decimal EstimatedValue { get; private set; }
    public string Period { get; private set; }

    public EarlyPainting(string title, int year, string dimensions,
        string location, decimal value, string style)
        : base(title, style)
    {
        if (year < 1900 || year > 1909)
            throw new ArgumentException("Ранният период е 1900-1909 г.");

        YearCreated = YearCreated < 0 ? 1900 : year;
        Dimensions = dimensions;
        Location = location;
    }
}

```

```

        EstimatedValue = value;
        Period = "Панен период (1900-1909)";
    }

    public override void Description()
    {
        Console.WriteLine($"↵\"{Title}\"\", ({YearCreated}) / {Period}.
{GetShortDescription()}");
    }

    public override void Details()
    {
        Console.WriteLine("=== Подробни характеристики ===");
        Console.WriteLine($"Заглавие: \"{Title}\"");
        Console.WriteLine($"Художник: {Artist}");
        Console.WriteLine($"Период: {Period}");
        Console.WriteLine($"Година на създаване: {YearCreated}");
        Console.WriteLine($"Размери: {Dimensions}");
        Console.WriteLine($"Местоположение: {Location}");
        Console.WriteLine($"Оценявана стойност: {EstimatedValue:C}");
        Console.WriteLine($"Стил: {Style}");
        Console.WriteLine($"Характеристики на периода:
{PeriodCharacteristics()}");
        Console.WriteLine();
    }

    public override string PeriodCharacteristics()
    {
        return "Синя и Розова епоха, меланхолия, социални теми, цигани,
артисти";
    }

    private string GetShortDescription()
    {
        return "Меланхолични тонове, социална тематика, емоционална дълбочина.";
    }
}

public class CubistPainting : Gallery
{
    public int YearCreated { get; private set; }
    public string Dimensions { get; private set; }
    public string Location { get; private set; }
    public decimal EstimatedValue { get; private set; }
    public string Period { get; private set; }
    public string CubistPhase { get; private set; } // Аналитичен/Синтетичен
кубизъм

    public CubistPainting(string title, int year, string dimensions,
string location, decimal value, string style,
string cubistPhase)
        : base(title, style)
    {
        if (year < 1910 || year > 1973)
            throw new ArgumentException("Кубистичният период е 1910-1973 г.");

        YearCreated = year;
        Dimensions = dimensions;

```



```

        Location = location;
        EstimatedValue = value;
        Period = "Кубистичен и по-късен период (1910-1973)";
        CubistPhase = cubistPhase;
    }

    public override void Description()
    {
        Console.WriteLine($">\{Title}\", ({YearCreated}) / {CubistPhase}
кубизъм. {GetShortDescription()}");
    }

    public override void Details()
    {
        Console.WriteLine("=== Подробни характеристики (Кубистичен период)
===");
        Console.WriteLine($"Заглавие: \{Title}\"");
        Console.WriteLine($"Художник: {Artist}");
        Console.WriteLine($"Период: {Period}");
        Console.WriteLine($"Фаза на кубизма: {CubistPhase}");
        Console.WriteLine($"Година на създаване: {YearCreated}");
        Console.WriteLine($"Размери: {Dimensions}");
        Console.WriteLine($"Местоположение: {Location}");
        Console.WriteLine($"Оценявана стойност: {EstimatedValue:C}");
        Console.WriteLine($"Стил: {Style}");
        Console.WriteLine($"Характеристики на периода:
{PeriodCharacteristics()}");
        Console.WriteLine();
    }

    public override string PeriodCharacteristics()
    {
        return "Деформация на форми, множествена перспектива, геометрични форми,
колаж";
    }

    private string GetShortDescription()
    {
        return "Иновативен подход, революция в изкуството, геометрични форми.";
    }
}

```